

ServiceMonitor vs. PodMonitor (Prometheus Operator)

Beide sind Custom Resources der API-Gruppe `monitoring.coreos.com/v1`, die vom Prometheus Operator ausgewertet werden, um automatisch Scrape-Konfigurationen für Prometheus zu generieren. Der Unterschied liegt darin, **worüber** die Scrape-Ziele gefunden werden.

1. Funktionsweise ServiceMonitor

`spec.selector` matcht Labels eines **Service**-Objekts. Der Prometheus Operator schaut sich anschließend die zum Service gehörenden `Endpoints/EndpointSlice` an, um die tatsächlichen Pod-IPs zu ermitteln. Es gibt also einen Umweg über den Service als Indirektionsschicht – genau wie beim normalen Traffic-Routing.

Wichtig: Nur Pods, die laut Readiness-Probe **Ready** sind, tauchen in den Endpoints auf und werden dadurch gescraped.

2. Funktionsweise PodMonitor

`spec.selector` matcht Labels direkt auf dem **Pod**. Kein Service nötig – Prometheus scrapst die Pod-IP unmittelbar, ohne Umweg über Endpoints. Der referenzierte Port (`podMetricsEndpoints[].port`) ist der benannte Container-Port im Pod-Spec, nicht ein Service-Port.

3. Vergleich

	ServiceMonitor	PodMonitor
Selector matcht	Labels des Service-Objekts	Labels des Pods direkt
Braucht einen Service?	Ja, zwingend	Nein
Discovery-Pfad	Service → Endpoints/EndpointSlice → Pod-IP	direkt Pod-IP
Scraped nur Ready-Pods?	Ja (Endpoints enthalten nur Ready-Pods)	Nein, i.d.R. alle laufenden Pods, unabhängig von Readiness
Port-Referenz	Name aus <code>Service.spec.ports[].name</code>	Name aus <code>Pod.spec.containers[].ports[].name</code>
Typischer Einsatz	Anwendungen, die ohnehin einen Service haben (Standardfall)	Anwendungen ohne Service, Sidecars, DaemonSets, Batch-Jobs, Pod-individuelle Metrik-Ports
Zusätzlicher Pflegeaufwand	Nutzt vorhandene Service-Labels weiter, keine zweite Label-Quelle	Eigene Pod-Label-Pflege nötig, falls sich diese von Service-Labels unterscheiden

Damit ein ServiceMonitor bzw. PodMonitor überhaupt von Prometheus berücksichtigt wird, muss sein **eigenes Label** zum `serviceMonitorSelector` bzw. `podMonitorSelector` der Prometheus-Custom-Resource im Cluster passen. Das ist unabhängig vom `spec.selector` innerhalb des Objekts selbst – ohne passendes Label wird das Objekt zwar valide im Cluster angelegt, aber von Prometheus schlicht ignoriert.

4. Implementierungsbeispiel ServiceMonitor

Variante mit gemeinsamem Service-Label (empfohlener Ansatz für Anwendungsgruppen mit gleichem Metrics-Endpoint):

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: servicemonitor-mkk-t1
  labels:
    kubernetes/name: servicemonitor-mkk
spec:
  selector:
    matchLabels:
      monitoring: prometheus
  endpoints:
    - port: <PORT_NAME>
      path: <METRICS_PATH>
      interval: <SCRAPE_INTERVAL>
```

Dazu auf jedem betroffenen Service das Label `monitoring: prometheus` setzen.

Alternative mit expliziter Werteliste (matcht mehrere app-Werte über einen ServiceMonitor, ohne gemeinsames Zusatz-Label):

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: servicemonitor-mkk-t1
  labels:
    kubernetes/name: servicemonitor-mkk
spec:
  selector:
    matchExpressions:
      - key: app
        operator: In
        values:
          - test1
          - test2
          - test3
  endpoints:
    - port: <PORT_NAME>
      path: <METRICS_PATH>
      interval: <SCRAPE_INTERVAL>
```

`matchLabels` kann pro Key nur einen Wert halten – `app: test1`, `app: test2`, `app: test3` gleichzeitig als `matchLabels` ist nicht möglich (letzter Wert gewinnt, YAML-Map-Key ist eindeutig). Für "matcht Wert A ODER B ODER C" ist `matchExpressions` mit `operator: In` nötig.

5. Implementierungsbeispiel PodMonitor

```

apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: <PODMONITOR_NAME>
  labels:
    kubernetes/name: <PODMONITOR_LABEL_NAME>
spec:
  selector:
    matchLabels:
      monitoring: prometheus
  podMetricsEndpoints:
    - port: <PORT_NAME_AM_POD>
      path: <METRICS_PATH>
      interval: <SCRAPE_INTERVAL>

```

Dazu auf jedem betroffenen Pod-Template (z. B. `Deployment.spec.template.metadata.labels`) das Label `monitoring: prometheus` setzen.

6. Entscheidungshilfe

- **Anwendung hat bereits einen Service** (Standardfall) → `ServiceMonitor`, nutzt die vorhandene Service-Label-Struktur weiter und scraped nur Ready-Pods.
- **Anwendung hat keinen Service** (Batch-Job, DaemonSet, Sidecar-Metrik) → `PodMonitor`, da hier keine Service-Indirektion existiert, über die `ServiceMonitor` arbeiten könnte.
- **Thematisch zusammengehörige Anwendungsgruppe mit identischem Metrics-Endpoint** → gemeinsames Label (z. B. `monitoring: prometheus`) auf allen betroffenen Services/Pods setzen, ein einziger `ServiceMonitor`/`PodMonitor` genügt für die ganze Gruppe.
- **Gruppen mit unterschiedlichem Port/Pfad/Interval** → pro Gruppe ein eigener `ServiceMonitor`/`PodMonitor` mit eigenem Label-Wert, da endpoints bzw. `podMetricsEndpoints` pro Objekt fix für alle gematchten Ziele gilt.